

Dobot Magician

Demo 说明

文档版本: V1.2

发布日期: 2021-08-16

版权所有 © 越疆科技有限公司2021。 保留一切权利。

非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

免责声明

在法律允许的最大范围内，本手册所描述的产品（含其硬件、软件、固件等）均“按照现状”提供，可能存在瑕疵、错误或故障，越疆不提供任何形式的明示或默示保证，包括但不限于适销性、质量满意度、适合特定目的、不侵犯第三方权利等保证；亦不对使用本手册或使用本公司产品导致的任何特殊、附带、偶然或间接的损害进行赔偿。

在使用本产品前详细阅读本使用手册及网上发布的相关技术文档并了解相关信息，确保在充分了解机器人及其相关知识的前提下使用机械臂。越疆建议您在专业人员的指导下使用本手册。该手册所包含的所有安全方面的信息都不得视为Dobot的保证，即便遵循本手册及相关说明，使用过程中造成的危害或损失依然有可能发生。

本产品的使用者有责任确保遵循相关国家的切实可行的法律法规，确保在越疆机械臂的使用中不存在任何重大危险。

越疆科技有限公司

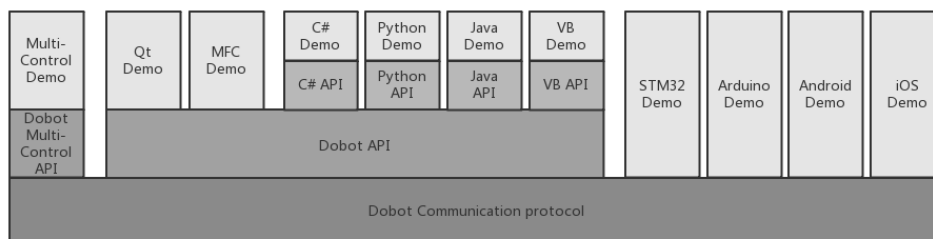
地址：深圳市南山区留仙大道3370号南山智园崇文区2号楼9-10楼

网址：<http://cn.dobot.cc/>

前言

前言

本说明文档旨在协助 Dobot Magician 二次开发者熟悉 Dobot 常用 API 和快速搭建开发环境。分别对多种语言，多种框架，多种系统的二次开发环境搭建和 Demo 代码进行说明。



修订记录

时间	修订记录
2018/06/30	第一次发布
2019/03/12	- 更新Java Demo中连接机械臂ConnectDobot方法的参数 - 更新C# Demo的界面显示图和获取位姿代码
2020/04/01	添加Python Demo版本说明
2021/08/16	更新QT版本下载链接

符号约定

在本手册中可能出现下列标志，它们所代表的含义如下。

符号	说明
 危险	表示有高度潜在危险，如果不能避免，会导致人员死亡或严重伤害
 警告	表示有中度或低度潜在危害，如果不能避免，可能导致人员轻微伤害、机械臂毁坏等情况
 注意	表示有潜在风险，如果忽视这些文本，可能导致机械臂损坏、数据丢失或不可预知的结果
 说明	表示是正文的附加信息，是对正文的强调和补充

目 录

1. 通用桌面系统	1
1.1 Dobot 动态链接库.....	1
1.1.1 编译	1
1.1.2 使用	1
1.2 Java Demo.....	1
1.2.1 工程说明	1
1.2.2 Java API.....	2
1.2.3 代码说明	2
1.3 MFC Demo	5
1.3.1 工程说明	5
1.3.2 代码说明	6
1.4 C# Demo	10
1.4.1 工程说明	10
1.4.2 C# API.....	10
1.4.3 代码说明	10
1.5 VB Demo	13
1.5.1 工程说明	13
1.5.2 VB API	14
1.5.3 代码说明	14
1.6 Qt Demo.....	16
1.6.1 工程说明	16
1.6.2 代码说明	16
1.7 Multi-Control Demo	19
1.7.1 工程说明	20
1.7.2 代码说明	20
1.8 Python Demo	23
1.8.1 工程说明	23
1.8.2 Python API	23
1.8.3 代码说明	24
2. 嵌入式系统	26
2.1 注意事项	26
2.2 STM32 Demo	26
2.2.1 硬件说明	26
2.2.2 工程说明	27
2.2.3 代码说明	28
2.3 Arduino Demo	30
2.3.1 硬件说明	30
2.3.2 工程说明	31
2.4 iOS Demo	32
2.4.1 工程说明	32
2.4.2 代码说明	33
2.4.3 代码说明	35

2.5	Android Demo	39
2.5.1	工程说明	39
2.5.2	代码说明	39

1. 通用桌面系统

对于通用桌面系统，我们已经为二次开发者提供了动态链接库。二次开发者只要直接调用动态链接库即可以控制Dobot Magician，而不需进行通讯协议相关的开发工作。

1.1 Dobot 动态链接库

在 DobotDll 文件夹下可以找到动态链接库的源码和预编译文件。其中，源码使用 Qt 5.6 开发环境。另外，文件夹中可以找到 Windows32 位、Windows64 位、Linux 和 Mac 四种平台的对应动态链接库。

1.1.1 编译

开发者请在 <https://download.qt.io/archive/qt/5.12/5.12.11/> 中下载适合自己系统的 QT 版本并安装。请使用 Qt5.12 重新编译 Dll 后，再使用 Demo。

如果项目需要配合qyqt库，请使用带MSVC编译器的Qt版本并用MSVC编译Dobot动态链接库。

1.1.2 使用

- Windows系统中，将动态链接库所在目录添加到系统Path环境变量。
- Linux系统中，在~/.bash_profile文件最后添加下列语句，并重启电脑。

程序 1.1 Linux 添加环境变量语句

```
export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:DOBOT_LIB_PATH
```

- Mac系统中，在~/.bash_profile文件最后添加下列语句，并重启电脑。

程序 1.2 Mac 添加环境变量语句

```
export DYLD_LIBRARY_PATH=$DYLD_LIBRARY_PATH: DOBOT_LIB_PATH
```

1.2 Java Demo

1.2.1 工程说明

配置环境：导入jna,用于Java直接访问本地动态库。

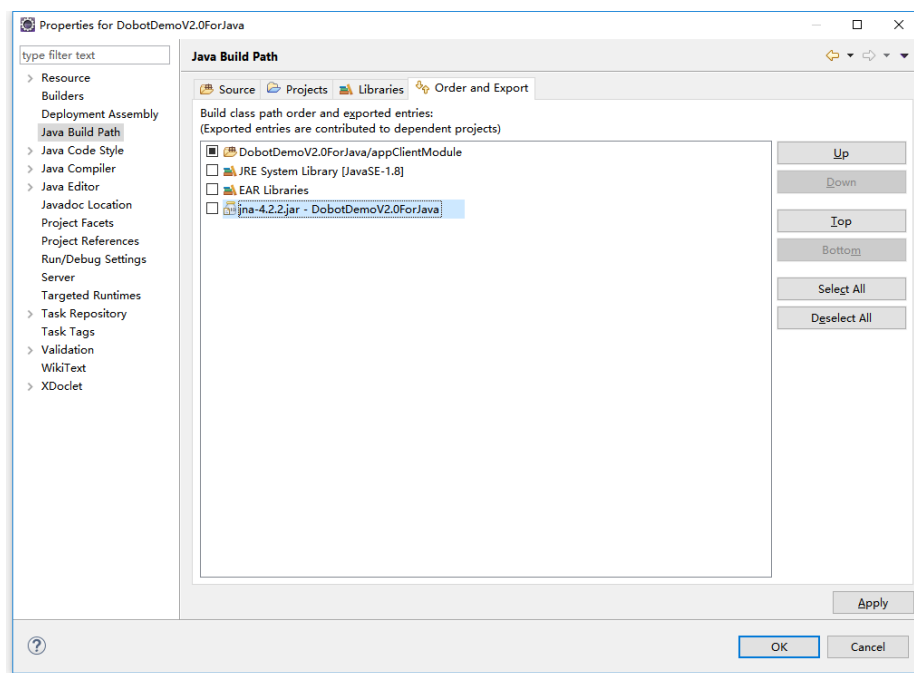


图 1.1 环境配置

1.2.2 Java API

DobotDll.java 是 Dobot 的 Java API 文件，将 Dobot 动态链接进行了二次封装。其中，加载动态链接库的代码如下：

程序 1.3 加载动态链接库

```
DobotDll instance = (DobotDll) Native.loadLibrary("DobotDll", DobotDll.class);
```

其中，“DobotDll”是 Windows 系统中动态链接库的文件名，用户需要根据不同的系统环境修改正确的动态链接库文件名。

1.2.3 代码说明

- (1) 连接机械臂，并且判断是否连接成功。

程序 1.4 连接机械臂并判断是否连接成功

```
IntByReference ib = new IntByReference();
DobotResult ret = DobotResult.values()[
    DobotDll.instance.ConnectDobot(
        (char)0, 115200, (char)0, (char)0
    )];

// 开始连接
if ( ret == DobotResult.DobotConnect_NotFound ||
    ret == DobotResult.DobotConnect_Occupied )
```

```
{  
    Msg("Connect error, code:" + ret.name());  
    return;  
}  
Msg("connect success code:" + ret.name());
```

- (2) 设置夹具中心距长度。

程序 1.5 设置夹具中心距长度

```
EndEffectorParams endEffectorParams = new EndEffectorParams();  
endEffectorParams.xBias = 71.6f;  
endEffectorParams.yBias = 0;  
endEffectorParams.zBias = 0;  
DobotDll.instance.SetEndEffectorParams(endEffectorParams, false, ib);
```

- (3) 设置关节坐标系点动速度。

程序 1.6 设置关节速度

```
JOGJointParams jogJointParams = new JOGJointParams();  
for(int i = 0; i < 4; i++) {  
    jogJointParams.velocity[i] = 200;  
    jogJointParams.acceleration[i] = 200;  
}  
DobotDll.instance.SetJOGJointParams(jogJointParams, false, ib);
```

- (4) 设置笛卡尔坐标系点动速度。

程序 1.7 设置坐标轴点动速度

```
JOGCoordinateParams jogCoordinateParams = new JOGCoordinateParams();  
for(int i = 0; i < 4; i++) {  
    jogCoordinateParams.velocity[i] = 200;  
    jogCoordinateParams.acceleration[i] = 200;  
}  
DobotDll.instance.SetJOGCoordinateParams(jogCoordinateParams, false, ib);
```

- (5) 设置示教再现速度与加速度百分比，如无设置，默认为 50%。

程序 1.8 设置示教再现速度与加速度的百分比

```
JOGCommonParams jogCommonParams = new JOGCommonParams();  
jogCommonParams.velocityRatio = 50;  
jogCommonParams.accelerationRatio = 50;
```



```
DobotDll.instance.SetJOGCommonParams(jogCommonParams, false, ib);
```

- (6) 设置示教再现的关节坐标系运动参数。

程序 1.9 设置示教再现的关节运动参数

```
PTPJointParams ptpJointParams = new PTPJointParams();
for(int i = 0; i < 4; i++) {
    ptpJointParams.velocity[i] = 200;
    ptpJointParams.acceleration[i] = 200;
}
DobotDll.instance.SetPTPJointParams(ptpJointParams, false, ib);
```

- (7) 设置示教再现的笛卡尔坐标系运动参数。

程序 1.10 设置示教再现的坐标系运动参数

```
PTPCoordinateParams ptpCoordinateParams = new PTPCoordinateParams();
ptpCoordinateParams.xyzVelocity = 200;
ptpCoordinateParams.xyzAcceleration = 200;
ptpCoordinateParams.rVelocity = 200;
ptpCoordinateParams.rAcceleration = 200;
DobotDll.instance.SetPTPCoordinateParams(ptpCoordinateParams, false, ib);
```

- (8) 设置示教再现中 JUMP 运动模式时的运动参数。

程序 1.11 设置示教再现中 JUMP 运动模式时的参数

```
PTPJumpParams ptpJumpParams = new PTPJumpParams();
ptpJumpParams.jumpHeight = 20;
ptpJumpParams.zLimit = 180;
DobotDll.instance.SetPTPJumpParams(ptpJumpParams, false, ib);
```

- (9) 获取机械臂的姿态信息。

程序 1.12 获取机械臂的姿态信息

```
Pose pose = new Pose();
DobotDll.instance.GetPose(pose);
Msg( "joint1Angle="+pose.jointAngle[0]+" "
    + "joint2Angle="+pose.jointAngle[1]+" "
    + "joint3Angle="+pose.jointAngle[2]+" "
    + "joint4Angle="+pose.jointAngle[3]+" "
    + "x="+pose.x+" "
    + "y="+pose.y+" "
```

```
+ "z="+pose.z+" "
+ "r="+pose.r+" ");
```

(10) 示教再现（两点之间来回运动）。

程序 1.13 示教再现

```
while(true)
{
    try{
        PTPCmd ptpCmd = new PTPCmd();
        ptpCmd.ptpMode = 0;
        ptpCmd.x = 260;
        ptpCmd.y = 0;
        ptpCmd.z = 50;
        ptpCmd.r = 0;
        DobotDll.instance.SetPTPCmd(ptpCmd, true, ib);
        //Thread.sleep(200);
        ptpCmd.ptpMode = 0;
        ptpCmd.x = 220;
        ptpCmd.y = 0;
        ptpCmd.z = 80;
        ptpCmd.r = 0;
        DobotDll.instance.SetPTPCmd(ptpCmd, true, ib);
    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

1.3 MFC Demo

1.3.1 工程说明

如图 1.2所示，三个功能模块依次为机械臂点动（JOG），获取姿态信息（Pose），示教再现（PTP）。

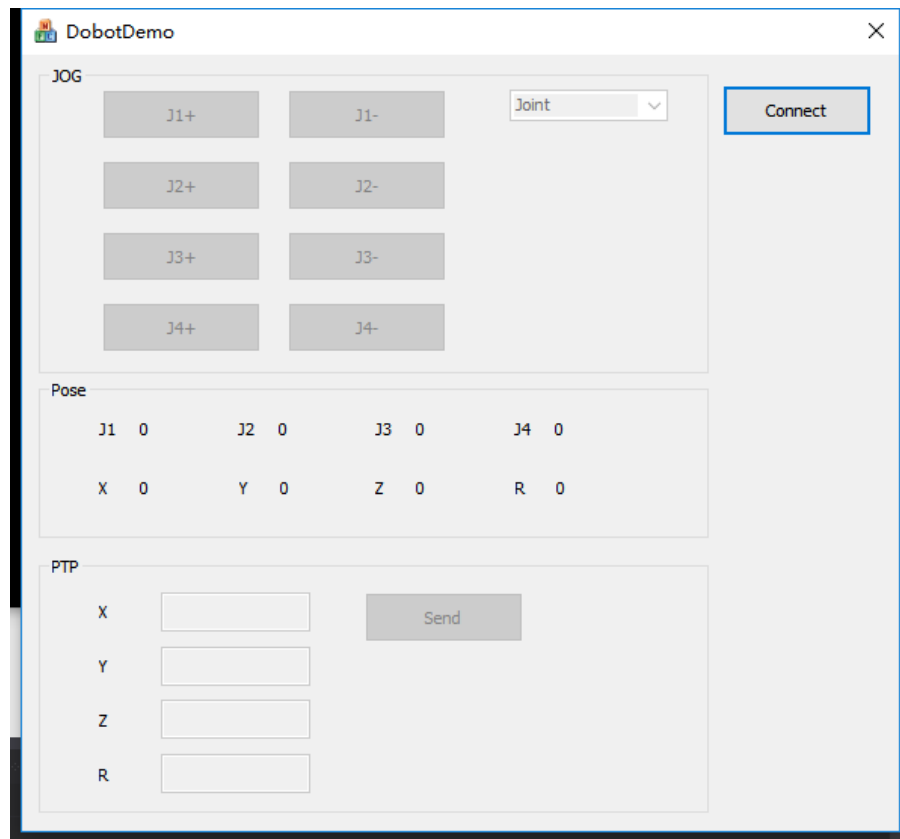


图 1.2 MFC Demo 界面

1.3.2 代码说明

- (1) 连接机械臂，并且判断机械臂是否成功。

程序 1.14 连接机械臂

```
if (!m_bConnectStatus) {
    if (ConnectDobot(0, 115200) != DobotConnect_NoError) {
        ::AfxMessageBox(L"Cannot connect Dobot!");
        return;
    }
}
```

- (2) 获取机械臂序列号。

程序 1.15 获取机械臂序列号

```
char deviceSN[64];
GetDeviceSN(deviceSN, sizeof(deviceSN));
```

- (3) 获取机械臂名称。

程序 1.16 获取机械臂名称

```
char deviceName[64];
```

```
GetDeviceName(deviceName, sizeof(deviceName));
```

- (4) 获取机械臂版本信息。

程序 1.17 获取机械臂版本信息

```
uint8_t majorVersion, minorVersion, revision;  
GetDeviceVersion(&majorVersion, &minorVersion, &revision);
```

- (5) 设置夹具中心距长度。

程序 1.18 设置夹具中心长度

```
EndEffectorParams endEffectorParams;  
memset(&endEffectorParams, 0, sizeof(EndEffectorParams));  
endEffectorParams.xBias = 71.6f;  
SetEndEffectorParams(&endEffectorParams, false, NULL);
```

- (6) 设置关节坐标系点动参数。

程序 1.19 设置关节坐标系点动参数

```
JOGJointParams jogJointParams;  
for (uint32_t i = 0; i < 4; i++) {  
    jogJointParams.velocity[i] = 200;  
    jogJointParams.acceleration[i] = 200;  
}  
SetJOGJointParams(&jogJointParams, false, NULL);
```

- (7) 设置笛卡尔坐标系点动参数。

程序 1.20 设置笛卡尔坐标系点动参数

```
JOGCoordinateParams jogCoordinateParams;  
for (uint32_t i = 0; i < 4; i++) {  
    jogCoordinateParams.velocity[i] = 200;  
    jogCoordinateParams.acceleration[i] = 200;  
}  
SetJOGCoordinateParams(&jogCoordinateParams, false, NULL);
```

- (8) 设置示教再现中的速度与加速度百分比，如无设置，默认为 50%。

程序 1.21 设置示教参数

```
JOGCommonParams jogCommonParams;  
jogCommonParams.velocityRatio = 50;  
jogCommonParams.accelerationRatio = 50;
```

```
SetJOGCommonParams(&jogCommonParams, false, NULL);
```

- (9) 设置示教再现的关节坐标系运动参数。

程序 1.22 设置示教关节运动参数

```
PTPJointParams ptpJointParams;  
for (uint32_t i = 0; i < 4; i++) {  
    ptpJointParams.velocity[i] = 200;  
    ptpJointParams.acceleration[i] = 200;  
}  
SetPTPJointParams(&ptpJointParams, false, NULL);
```

- (10) 设置示教再现的笛卡尔坐标系运动参数。

程序 1.23 设置示教轴运动参数

```
PTPCoordinateParams ptpCoordinateParams;  
ptpCoordinateParams.xyzVelocity = 200;  
ptpCoordinateParams.xyzAcceleration = 200;  
ptpCoordinateParams.rVelocity = 200;  
ptpCoordinateParams.rAcceleration = 200;  
SetPTPCoordinateParams(&ptpCoordinateParams, false, NULL);
```

- (11) 设置示教再现中 JUMP 运动模式时的运动参数。。

程序 1.24 设置示教 Jump 参数

```
PTPJumpParams ptpJumpParams;  
ptpJumpParams.jumpHeight = 10;  
ptpJumpParams.zLimit = 150;  
SetPTPJumpParams(&ptpJumpParams, false, NULL);
```

- (12) 点动机械臂点动。

程序 1.25 机械点动

```
JOGCmd jogCmd;  
jogCmd.isJoint = m_JOGMode.GetCurSel() == 0;  
jogCmd.cmd = i + 1;  
SetJOGCmd(&jogCmd, false, NULL);
```

- (13) 获取机械臂姿态信息。

程序 1.26 获取状态信息

```
Pose pose;
```

```
if (GetPose(&pose) != DobotCommunicate_NoError) {  
    break;  
}  
  
CString str;  
str.Format(L"%1.3f", pose.jointAngle[0]);  
m_StaticJ1.SetWindowText(str);  
str.Format(L"%1.3f", pose.jointAngle[1]);  
m_StaticJ2.SetWindowText(str);  
str.Format(L"%1.3f", pose.jointAngle[2]);  
m_StaticJ3.SetWindowText(str);  
str.Format(L"%1.3f", pose.jointAngle[3]);  
m_StaticJ4.SetWindowText(str);  
str.Format(L"%1.3f", pose.x);  
m_StaticX.SetWindowText(str);  
str.Format(L"%1.3f", pose.y);  
m_StaticY.SetWindowText(str);  
str.Format(L"%1.3f", pose.z);  
m_StaticZ.SetWindowText(str);  
str.Format(L"%1.3f", pose.r);  
m_StaticR.SetWindowText(str);
```

(14) 机械臂示教再现运动 (PTP)。

程序 1.27 示教再现

```
PTPCmd ptpCmd;  
ptpCmd.ptpMode = mode;  
ptpCmd.x = x;  
ptpCmd.y = y;  
ptpCmd.z = z;  
ptpCmd.r = r;  
uint64_t queuedCmdIndex;  
do {  
    int result = SetPTPCmd(&ptpCmd, true, &queuedCmdIndex);  
    if (result == DobotCommunicate_NoError) {  
        break;  
    }  
} while (1);
```

1.4 C# Demo

1.4.1 工程说明

如图 1.3 所示，三个功能模块依次为机械臂点动（JOG），获取姿态信息（Pose），示教再现（Playback）。

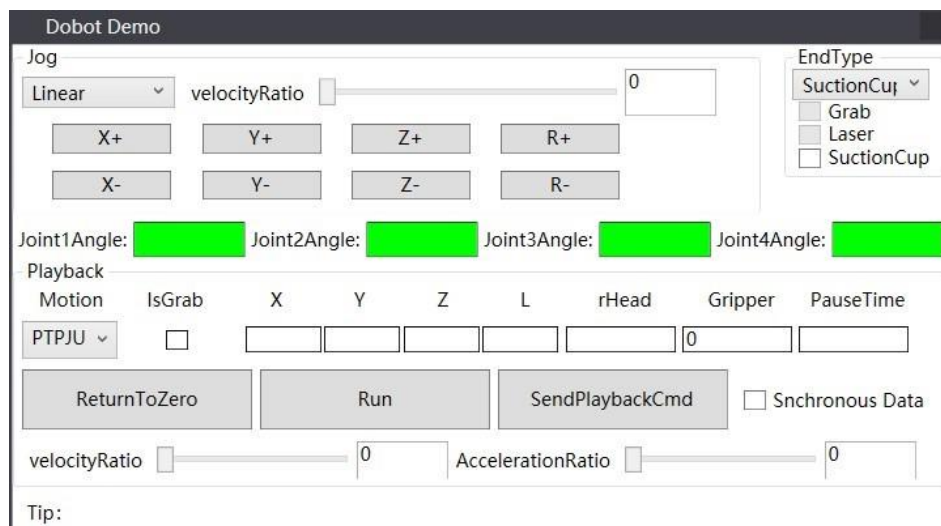


图 1.3 C# Demo 界面

1.4.2 C# API

DobotDll.cs与DobotDllType.cs是Dobot动态链接库的C# API文件，将Dobot动态链接库进行了二次封装。例如，连接函数（ConnectDobot）：

程序 1.28 连接函数

```
[DllImport("DobotDll.dll",
    EntryPoint = "ConnectDobot",
    CallingConvention = CallingConvention.Cdecl
)]

public static extern int ConnectDobot(string portName,
                                     int baudrate,
                                     StringBuilder fwType,
                                     StringBuilder version);
```

代码中的“DobotDll.dll”是Windows系统中的动态链接库文件名，用户需要根据自身系统修改正确的文件名。

1.4.3 代码说明

- （1）连接机械臂，并判断是否连接成功。

程序 1.29 连接机械臂

```
int ret = DobotDll.ConnectDobot("", 115200, fwType, version);
```

```
if (ret != (int)DobotConnect.DobotConnect_NoError)
{
    Msg("Connect error", MsgInfoType.Error);
    return;
}
```

- (2) 设置机械臂关节坐标系点动参数。

程序 1.30 设置关节坐标系点动参数

```
JOGJointParams jsParam;
jsParam.velocity = new float[] { 200, 200, 200, 200 };
jsParam.acceleration = new float[] { 200, 200, 200, 200 };
DobotDll.SetJOGJointParams(ref jsParam, false, ref cmdIndex);
```

- (3) 设置机械臂点动速度与加速度百分比。

程序 1.31 设置点动速度与加速度百分比

```
JOGCommonParams jdParam;
jdParam.velocityRatio = 100;
jdParam.accelerationRatio = 100;
DobotDll.SetJOGCommonParams(ref jdParam, false, ref cmdIndex);
```

- (4) 设置示教再现的关节坐标系运动参数。

程序 1.32 设置示教关节坐标系运动参数

```
PTPJointParams pbsParam;
pbsParam.velocity = new float[] { 200, 200, 200, 200 };
pbsParam.acceleration = new float[] { 200, 200, 200, 200 };
DobotDll.SetPTPJointParams(ref pbsParam, false, ref cmdIndex);
```

- (5) 设置示教再现的笛卡尔坐标系运动参数。

程序 1.33 设置示教坐标运动参数

```
PTPCoordinateParams cpbsParam;
cpbsParam.xyzVelocity = 100;
cpbsParam.xyzAcceleration = 100;
cpbsParam.rVelocity = 100;
cpbsParam.rAcceleration = 100;
DobotDll.SetPTPCoordinateParams(ref cpbsParam, false, ref cmdIndex);
```

- (6) 设置示教再现中 JUMP 运动模式时的运动参数。

程序 1.34 设置示教 Jump 运动参数

```
PTPJumpParams pjp;  
pjp.jumpHeight = 20;  
pjp.zLimit = 100;  
DobotDll.SetPTPJumpParams(ref pjp, false, ref cmdIndex);
```

- (7) 设置示教再现速度与加速度百分比。

程序 1.35 设置示教参数

```
PTPCommonParams pbdParam;  
pbdParam.velocityRatio = 30;  
pbdParam.accelerationRatio = 30;  
DobotDll.SetPTPCommonParams(ref pbdParam, false, ref cmdIndex);
```

- (8) 点动机械臂 (JOG)。

程序 1.36 点动机械臂

```
currentCmd.isJoint = isJoint;  
currentCmd.cmd = e.ButtonState == MouseButtonState.Pressed ?  
    (byte)JogCmdType.JogAPressed :  
    (byte)JogCmdType.JogIdle;  
DobotDll.SetJOGCmd(ref currentCmd, false, ref cmdIndex);
```

- (9) 获取机械臂姿态信息 (Pose)。

程序 1.37 获取位姿

```
DobotDll.GetPose(ref pose);  
DobotDll.GetPoseL(ref PoseL);  
this.Dispatcher.BeginInvoke((Action)delegate()  
{  
    tbJoint1Angle.Text = pose.jointAngle[0].ToString();  
    tbJoint2Angle.Text = pose.jointAngle[1].ToString();  
    tbJoint3Angle.Text = pose.jointAngle[2].ToString();  
    tbJoint4Angle.Text = pose.jointAngle[3].ToString();  
    if (sync.IsChecked == true)  
    {  
        X.Text = pose.x.ToString();  
        Y.Text = pose.y.ToString();  
        Z.Text = pose.z.ToString();  
        L.Text = PoseL.ToString();  
    }  
});
```

```
        rHead.Text = pose.rHead.ToString();  
        pauseTime.Text = "0";  
    }  
});
```

(10) 示教再现运动 (PTP)。

程序 1.38 示教再现

```
pdbCmd.ptpMode = style;  
pdbCmd.x = x;  
pdbCmd.y = y;  
pdbCmd.z = z;  
pdbCmd.rHead = r;  
while(true)  
{  
    int ret = DobotDll.SetPTPCmd(ref pdbCmd, true, ref cmdIndex);  
    if (ret == 0)  
        break;  
}
```

(11) 获取机械臂报警信息 (Alarm)。

程序 1.39 获取报警信息

```
int ret;  
byte[] alarmsState = new byte[32];  
UInt32 len = 32;  
ret = DobotDll.GetAlarmsState(alarmsState, ref len, alarmsState.Length);
```

1.5 VB Demo

1.5.1 工程说明

该 Demo 简单展示了连接机械臂后，用示教再现方式分别运动到两个坐标点 (PTP1 与 PTP2)，如图 1.4 所示。

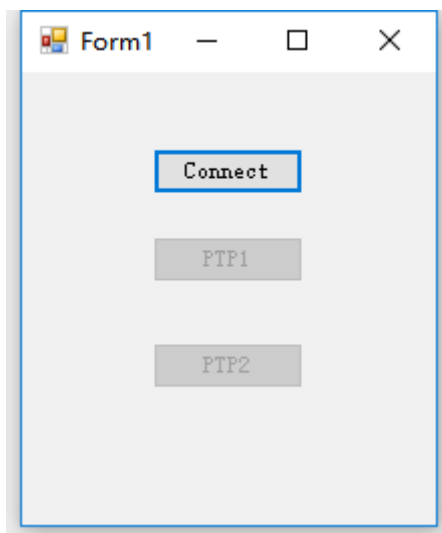


图 1.4 VB Demo 界面

1.5.2 VB API

DobotDll.vb与DobotDllType.vb是Dobot的VB API文件，将Dobot动态链接库进行了二次封装。例如，连接机械臂函数（ConnectDobot）：

程序 1.40 连接函数

```
Class DobotDll
<DllImport("DobotDll.dll", CallingConvention:=CallingConvention.Cdecl)> Public Shared Function
ConnectDobot(ByVal portName As String, ByVal baudrate As Int32) As Int32
End Function
End Class
```

代码中的“DobotDll.dll”是Windows系统中的动态链接库文件名，用户需要根据自身系统环境修改为正确的文件名。

1.5.3 代码说明

（1） 连接机械臂。

程序 1.41 连接机械臂

```
result = DobotDll.ConnectDobot("", 115200)
If result <> 0 Then
    MsgBox("Could not find Dobot or Dobot is occupied!")
    Return
End If
```

（2） 获取机械臂信息。

程序 1.42 获取机械臂信息

```
DobotDll.GetDeviceName(deviceName, 64)
```

(3) 机械臂示教再现运动 (PTP)。

程序 1.43 示教再现

```
Dim ptpCmd As PTPCmd
ptpCmd.ptpMode = ptpMode
ptpCmd.x = x
ptpCmd.y = y
ptpCmd.z = z
ptpCmd.r = r
Dim result As Int32
Dim queuedCmdIndex As UInt64
Dim currentQueuedCmdIndex As UInt64
While True
    result = DobotDll.SetPTPCmd(ptpCmd, True, queuedCmdIndex)
    If result = DobotCommunicate.DobotCommunicate_NoError Then
        Exit While
    End If
End While
```

(4) 获取机械臂姿态信息。

程序 1.44 获取机械臂姿态

```
result = DobotDll.GetPose(pose)
If result <> DobotCommunicate.DobotCommunicate_NoError Then
    Return
End If
Debug.Print(pose.x)
Debug.Print(pose.y)
Debug.Print(pose.z)
Debug.Print(pose.r)
Debug.Print(pose.joint1Angle)
Debug.Print(pose.joint2Angle)
Debug.Print(pose.joint3Angle)
Debug.Print(pose.joint4Angle)
```

1.6 Qt Demo

1.6.1 工程说明

请选择Qt5.0及以上的版本。当QtCreator的编译器为mingw时，直接链接DobotDll.dll即可。如果编译器为MSVC时，需要在DobotDll.dll所在的库文件夹中添加DobotDll.lib。三个功能模块依次为机械臂点动（JOG），获取姿态信息（Pose），示教再现（Playback），如图 1.5 所示。

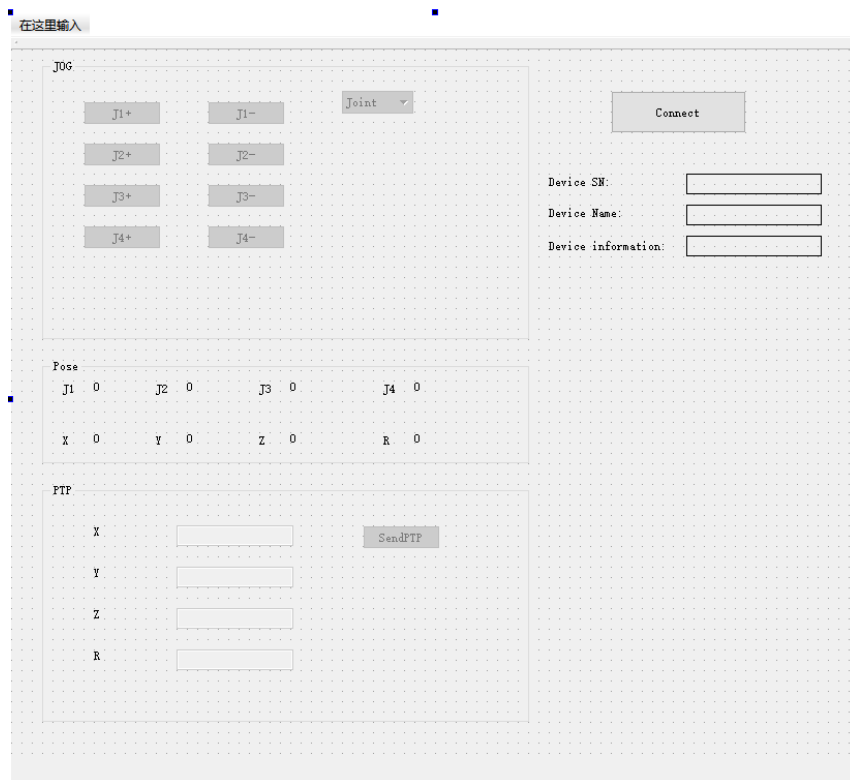


图 1.5 QT Demo 界面

1.6.2 代码说明

- (1) 连接机械臂，并判断是否连接成功。

程序 1.45 连接机械臂

```
if (!connectStatus) {  
    if (ConnectDobot(0, 115200) != DobotConnect_NoError) {  
        QMessageBox::information(this, tr("error"),  
                                tr("Connect dobot error!!!"),  
                                QMessageBox::Ok);  
        return;  
    }  
}
```

- (2) 获取机械臂序列号信息。

程序 1.46 获取机械臂序列号

```
char deviceSN[64];
GetDeviceSN(deviceSN, sizeof(deviceSN));
ui->deviceSNLabel->setText(deviceSN);
```

- (3) 获取机械臂名称。

程序 1.47 获取名字

```
char deviceName[64];
GetDeviceName(deviceName, sizeof(deviceName));
ui->DeviceNameLabel->setText(deviceName);
```

- (4) 获取机械臂版本信息。

程序 1.48 获取机械臂版本

```
uint8_t majorVersion, minorVersion, revision;
GetDeviceVersion(&majorVersion, &minorVersion, &revision);
ui->DeviceInfoLabel->setText(QString::number(majorVersion) + "." + QString::number(minorVersion) + "." +
QString::number(revision));
```

- (5) 设置机械臂夹具中心距参数。

程序 1.49 获取夹具中心距

```
EndEffectorParams endEffectorParams;
memset(&endEffectorParams, 0, sizeof(endEffectorParams));
endEffectorParams.xBias = 71.6f;
SetEndEffectorParams(&endEffectorParams, false, NULL);
```

- (6) 设置机械臂关节坐标系点动参数。

程序 1.50 设置关节坐标系点动参数

```
JOGJointParams jogJointParams;
for (int i = 0; i < 4; i++) {
    jogJointParams.velocity[i] = 100;
    jogJointParams.acceleration[i] = 100;
}
SetJOGJointParams(&jogJointParams, false, NULL);
```

- (7) 设置机械臂笛卡尔坐标系点动参数。

程序 1.51 设置坐标点动参数

```
JOGCoordinateParams jogCoordinateParams;  
for (int i = 0; i < 4; i++) {  
    jogCoordinateParams.velocity[i] = 100;  
    jogCoordinateParams.acceleration[i] = 100;  
}  
SetJOGCoordinateParams(&jogCoordinateParams, false, NULL);
```

- (8) 设置机械臂点动速度与加速度百分比，默认为 50。

程序 1.52 设置点动参数

```
JOGCommonParams jogCommonParams;  
jogCommonParams.velocityRatio = 50;  
jogCommonParams.accelerationRatio = 50;  
SetJOGCommonParams(&jogCommonParams, false, NULL);
```

- (9) 设置示教再现的关节坐标系运动参数。

程序 1.53 设置示教关节坐标系参数

```
PTPJointParams ptpJointParams;  
for (int i = 0; i < 4; i++) {  
    ptpJointParams.velocity[i] = 100;  
    ptpJointParams.acceleration[i] = 100;  
}  
SetPTPJointParams(&ptpJointParams, false, NULL);
```

- (10) 设置示教再现的笛卡尔坐标系运动参数。

程序 1.54 设置示教笛卡尔坐标系参数

```
PTPCoordinateParams ptpCoordinateParams;  
ptpCoordinateParams.xyzVelocity = 100;  
ptpCoordinateParams.xyzAcceleration = 100;  
ptpCoordinateParams.rVelocity = 100;  
ptpCoordinateParams.rAcceleration = 100;  
SetPTPCoordinateParams(&ptpCoordinateParams, false, NULL);
```

- (11) 设置示教再现中 Jump 运动模式时的运动参数。

程序 1.55 设置示教 Jump 参数

```
PTPJumpParams ptpJumpParams;  
ptpJumpParams.jumpHeight = 20;
```

```
ptpJumpParams.zLimit = 150;  
SetPTPJumpParams(&ptpJumpParams, false, NULL);
```

(12) 点动机械臂 (JOG)。

程序 1.56 点动

```
JOGCmd jogCmd;  
jogCmd.isJoint = ui->teachMode->currentIndex() == 0;  
jogCmd.cmd = index + 1;  
while (SetJOGCmd(&jogCmd, false, NULL) != DobotCommunicate_NoError)  
{...}
```

(13) 获取机械臂姿态信息 (Pose);

程序 1.57 获取机械臂位姿

```
Pose pose;  
while (GetPose(&pose) != DobotCommunicate_NoError) {...}  
ui->joint1Label->setText(QString::number(pose.jointAngle[0]));  
ui->joint2Label->setText(QString::number(pose.jointAngle[1]));  
ui->joint3Label->setText(QString::number(pose.jointAngle[2]));  
ui->joint4Label->setText(QString::number(pose.jointAngle[3]));  
ui->xLabel->setText(QString::number(pose.x));  
ui->yLabel->setText(QString::number(pose.y));  
ui->zLabel->setText(QString::number(pose.z));  
ui->rLabel->setText(QString::number(pose.r));
```

(14) 机械臂示教再现运动 (PTP)。

程序 1.58 示教再现

```
PTPCmd ptpCmd;  
ptpCmd.ptpMode = PTPMOVJXYZMode;  
ptpCmd.x = ui->xPTPEdit->text().toFloat();  
ptpCmd.y = ui->yPTPEdit->text().toFloat();  
ptpCmd.z = ui->zPTPEdit->text().toFloat();  
ptpCmd.r = ui->rPTPEdit->text().toFloat();  
while (SetPTPCmd(&ptpCmd, true, NULL) != DobotCommunicate_NoError)  
{...}
```

1.7 Multi-Control Demo

1.7.1 工程说明

此Demo的DobotDll库为专为多机联动所编译的函数库，与其他Demo的函数库不通用，不可混淆。

1.7.2 代码说明

该Demo代码与QtDemo的代码几乎一样，但是每个API函数增加了一个dobotId参数，以确定连接的机械臂ID号，用于多机联动。

- (1) 连接机械臂，动态库将返回连接的机械臂的 ID。后续对于机械臂的操作，都需要添加 ID 指定机械臂。

程序 1.59 连接机械臂

```
if (!connectStatus) {
    if (ConnectDobot(ui->lineEdit->text().toLatin1().data(), 115200, fwType, version,
        &dobotId) !=DobotConnect_NoError)
    {
        QMessageBox::information(this, tr("error"), tr("Connect dobot error!!!"), QMessageBox::Ok);
        return;
    }
}
qDebug() << "dobotId" << dobotId;
```

- (2) 获取机械臂序列号信息。

程序 1.60 获取机械臂序列号

```
char deviceSN[64];
GetDeviceSN(dobotId, deviceSN, sizeof(deviceSN));
ui->deviceSNLabel->setText(deviceSN);
```

- (3) 获取机械臂名称。

程序 1.61 获取机械臂名称

```
char deviceName[64];
GetDeviceName(dobotId, deviceName, sizeof(deviceName));
ui->DeviceNameLabel->setText(deviceName);
```

- (4) 获取机械臂版本信息。

程序 1.62 获取机械臂版本

```
uint8_t majorVersion, minorVersion, revision;
GetDeviceVersion(dobotId, &majorVersion, &minorVersion, &revision);
ui->DeviceInfoLabel->setText(QString::number(majorVersion) + "." + QString::number(minorVersion) + "." +
    QString::number(revision));
```

- (5) 设置机械臂夹具中心距参数。

程序 1.63 获取中心距

```
EndEffectorParams endEffectorParams;  
memset(&endEffectorParams, 0, sizeof(endEffectorParams));  
endEffectorParams.xBias = 71.6f;  
SetEndEffectorParams(dobotId, &endEffectorParams, false, NULL);
```

- (6) 设置机械臂关节坐标系点动运动参数。

程序 1.64 设置关节点动参数

```
JOGJointParams jogJointParams;  
for (int i = 0; i < 4; i++) {  
    jogJointParams.velocity[i] = 100;  
    jogJointParams.acceleration[i] = 100;  
}  
SetJOGJointParams(dobotId, &jogJointParams, false, NULL);
```

- (7) 设置机械臂笛卡尔坐标系点动运动参数。

程序 1.65 设置坐标点动参数

```
JOGCoordinateParams jogCoordinateParams;  
for (int i = 0; i < 4; i++) {  
    jogCoordinateParams.velocity[i] = 100;  
    jogCoordinateParams.acceleration[i] = 100;  
}  
SetJOGCoordinateParams(dobotId, &jogCoordinateParams, false, NULL);
```

- (8) 设置机械臂点动速度与加速度百分比，默认为 50%。

程序 1.66 设置点动速度和加速度百分比

```
JOGCommonParams jogCommonParams;  
jogCommonParams.velocityRatio = 50;  
jogCommonParams.accelerationRatio = 50;  
SetJOGCommonParams(dobotId, &jogCommonParams, false, NULL);
```

- (9) 设置示教再现的关节坐标系运动参数。

程序 1.67 设置示教关节坐标系运动参数

```
PTPJointParams ptpJointParams;  
for (int i = 0; i < 4; i++) {  
    ptpJointParams.velocity[i] = 100;
```

```
    ptpJointParams.acceleration[i] = 100;
}
SetPTPJointParams(dobotId, &ptpJointParams, false, NULL);
```

- (10) 设置示教再现的笛卡尔坐标系运动参数。

程序 1.68 设置示教笛卡尔坐标系运动参数

```
PTPCoordinateParams ptpCoordinateParams;
ptpCoordinateParams.xyzVelocity = 100;
ptpCoordinateParams.xyzAcceleration = 100;
ptpCoordinateParams.rVelocity = 100;
ptpCoordinateParams.rAcceleration = 100;
SetPTPCoordinateParams(dobotId, &ptpCoordinateParams, false, NULL);
```

- (11) 设置示教再现中 JUM 运动模式时运动参数。

程序 1.69 设置示教 Jump 参数

```
PTPJumpParams ptpJumpParams;
ptpJumpParams.jumpHeight = 20;
ptpJumpParams.zLimit = 150;
SetPTPJumpParams(dobotId, &ptpJumpParams, false, NULL);
```

- (12) 点动机械臂 (JOG)。

程序 1.70 点动机械臂

```
JOGCmd jogCmd;
jogCmd.isJoint = ui->teachMode->currentIndex() == 0;
jogCmd.cmd = index + 1;
while (SetJOGCmd(dobotId, &jogCmd, false, NULL) !=
        DobotCommunicate_NoError)
{...}
```

- (13) 获取机械臂姿态信息 (Pose)。

程序 1.71 获取机械臂位姿

```
Pose pose;
while (GetPose(dobotId, &pose) != DobotCommunicate_NoError) {
}
ui->joint1Label->setText(QString::number(pose.jointAngle[0]));
ui->joint2Label->setText(QString::number(pose.jointAngle[1]));
ui->joint3Label->setText(QString::number(pose.jointAngle[2]));
```

```
ui->joint4Label->setText(QString::number(pose.jointAngle[3]));
ui->xLabel->setText(QString::number(pose.x));
ui->yLabel->setText(QString::number(pose.y));
ui->zLabel->setText(QString::number(pose.z));
ui->rLabel->setText(QString::number(pose.r));
```

(14) 机械臂示教再现运动 (PTP)。

程序 1.72 示教再现

```
PTPCmd ptpCmd;
ptpCmd.ptpMode = PTPMOVJXYZMode;
ptpCmd.x = ui->xPTPEdit->text().toFloat();
ptpCmd.y = ui->yPTPEdit->text().toFloat();
ptpCmd.z = ui->zPTPEdit->text().toFloat();
ptpCmd.r = ui->rPTPEdit->text().toFloat();
SetPTPCmd(dobotId, &ptpCmd, true, NULL);
```

1.8 Python Demo

1.8.1 工程说明

Python Demo包括DobotControl.py, DobotDllType.py共两个文件。其中, DobotDllType.py是Dobot API的python二次封装, DobotControl是具体的实现文件。

运行DobotControl.py前需将Dobot动态链接库放到python的运行目录, 或者将Dobot动态链接库所在路径添加到系统环境变量。

不同版本Python需对应相应的版本demo, 例如64位Python应使用64位Python demo, 32位Python应使用32位Python demo, 否则程序将报错。

1.8.2 Python API

DobotDllType.py是Dobot的Python API文件, 对Dobot的动态链接库进行了二次封装。其中加载动态链接库的代码如程序 1.73。

程序 1.73 加载动态链接库

```
def load():
    if platform.system() == "Windows":
        print("您用的 dll 是 32 位, 为了顺利运行, 请保证您的 python 环境也是 32 位")
        print("python 环境是: ", platform.architecture())
        return CDLL("./DobotDll.dll", RTLD_GLOBAL)
    elif platform.system() == "Darwin":
        return CDLL("./libDobotDll.dylib", RTLD_GLOBAL)
    elif platform.system() == "Linux":
        return cdll.loadLibrary("libDobotDll.so")
```

**注意**

为保证正确加载动态链接库，请务必将Dobot动态链接库目录添加到系统环境变量中，详情请见1.1.2 使用。

1.8.3 代码说明

本例程中所有运动API均使用队列模式。

- (1) 加载动态链接库，获取库对象（api）。后续所有 python API 调用时都需使用到该对象。

程序 1.74 加载动态链接库

```
api = dType.load()
```

- (2) 连接 Dobot，并打印连接信息，连接成功才处理相关代码。

程序 1.75 连接 Dobot

```
state = dType.ConnectDobot(api, "", 115200)[0]
print("Connect status:", CON_STR[state])
if (state == dType.DobotConnect.DobotConnect_NoError):
    #Dobot 交互代码
dType.DisconnectDobot(api)
```

- (3) 队列控制(清空队列，开始队列，停止队列)。

程序 1.76 队列控制

```
dType.SetQueuedCmdClear(api)
dType.SetQueuedCmdStartExec(api)
#将运动指令下发到队列中
dType.SetQueuedCmdStopExec(api)
```

- (4) 设置运动参数。

程序 1.77 设置运动参数

```
dType.SetHOMEParams(api, 200, 200, 200, 200, isQueued = 1)
dType.SetPTPJointParams(api, 200, 200, 200, 200,
                        200, 200, 200, 200, isQueued = 1)
dType.SetPTPCommonParams(api, 100, 100, isQueued = 1)
```

- (5) 将来回 PTP 运动指令下发到队列中，并获取最后一条指令的 index。

程序 1.78 PTP 运动

```
for i in range(0, 5):
    if i % 2 == 0:
```

```
        offset = 50
    else:
        offset = -50
    lastIndex = dType.SetPTPCmd(api, dType.PTPMode.PTPMOVLXYZMode, 200 + offset, offset,
                                offset, offset, isQueued = 1)[0]
```

(6) 等待最后一条运动指令完成。

程序 1.79 等待最后一条运动指令完成

```
while lastIndex > dType.GetQueuedCmdCurrentIndex(api)[0]:
    dType.dSleep(100)
```

2. 嵌入式系统

对于嵌入式系统，用户需要根据Dobot通讯协议，进行通讯相关的开发工作。

2.1 注意事项

- DobotMagician扩展接口的电平信号是3.3V，耐压可达5V。
- 使用AD功能时不能接入超过3.3V的电压，其余的端口可以接入5V信号（包括串口接口）。
- 在使用除STM32和Arduino以外的芯片进行二次开发时，需要注意电平兼容。

2.2 STM32 Demo

2.2.1 硬件说明

本Demo是基于STM32F103VET6芯片编写，因此使用本程序时需要用户自行配备一块STM32F103VET6开发板（使用其他型号的芯片需要自行移植）。

该 Demo 中，通讯的端口使用了机械臂扩展 10PIN 接口，接口的类型为 FC-10P，接口定义如下图 2.1 所示。该 Demo 中，我们需要使用 RX、TX、GND 三个端口，机械臂与开发板连接对应关系：RX->TX1，TX->RX1，GND->GND 所示。

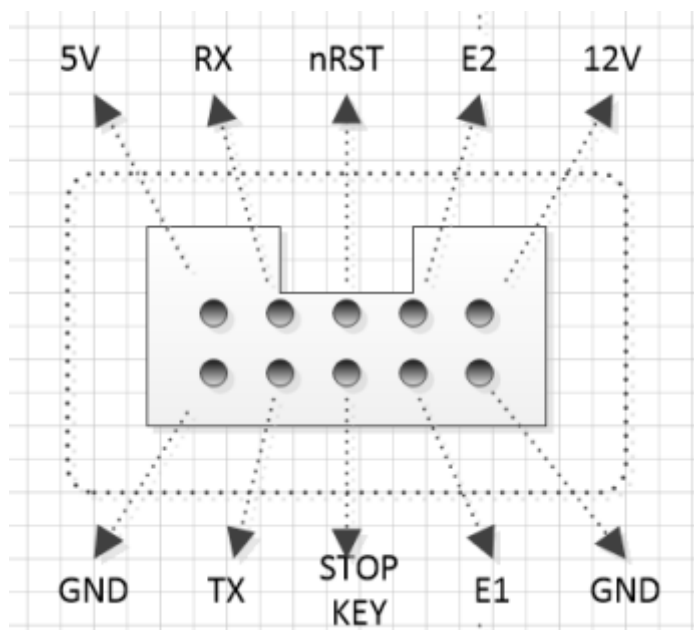


图 2.1 机械臂的扩展接口定义

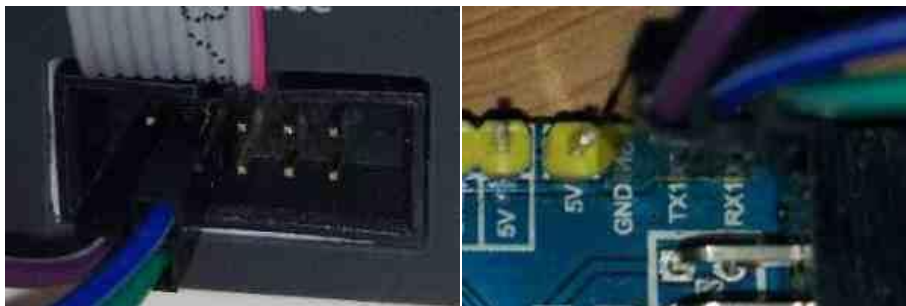


图 2.2 机械臂与开发板连接

2.2.2 工程说明

本工程使用的编译器软件为KEIL（4或5），DFP版本为2.0。

（1）通讯

Dobot协议指令由包头、负载帧长、负载帧、校验组成。

- 两个0xAA包头
- 参数长度（取值为2+N Params的长度）
- 命令索引ID
- Ctrl位（包括RW读写标记以及isQueued队列标记）
- 命令参数Params（不定长N）
- 检验和Checksum

通信协议详细内容请参见《Dobot Magician通信协议》。

表 2.1 通讯协议格式

Header	Len	Payload				Checksum
		ID	Ctrl		Params	
			rw	isQueued		
0XAA 0XAA	2+N	XX	1/0	1/0	N（Byte）	Payload Checksum

Dobot控制器支持两种类型的指令：立即指令与队列指令。

- 立即指令：Dobot控制器在收到指令后立即处理该指令，而不管当前控制器是否在还在处理其他指令。
- 队列指令：Dobot控制器在收到指令后会该指令放入控制器内部的指令队列中，Dobot控制器将顺序执行指令。

（2）文件结构

该 Demo 工程文件包含：app、dirver、core、stlib、stm32f10x、complatform 等 5 个目录。

- app：存放主要的命令和main函数。

- driver: 存放硬件的驱动文件，用于芯片的串口配置以及时钟配置。
- core: 存放M3的核心文件，一般不需要改动。
- Stlib: 库文件，一般不需要改动。
- stm32f10x: 库文件，一般不需要改动。
- complatform: 存放协议的处理文件，用户最好不要进行改动。

2.2.3 代码说明

(1) ProtocolProcess 函数说明

该Demo的发送和接收命令均保存在Ringbuffer里面，通过ProtocolProcess函数来处理命令数据。

(2) Common 命令解析

用户需处理的文件是main.cpp文件和common.cpp文件。main.cpp是主函数文件，common是命令处理文件。以典型的PTP命令来说明，SetPTPCmd函数传入的一个PTPCmd的结构体数据，队列标记，以及索引编号（暂未使用，主要用来记录当前下发的命令的序列编号）。

程序 2.1 SetPTPCmd 接口

```
int SetPTPCmd(PTPCmd *ptpCmd, bool isQueued,
              uint64_t *queuedCmdIndex)
{
    Message tempMessage;
    memset(&tempMessage, 0, sizeof(Message));
    tempMessage.id = ProtocolPTPCmd;
    tempMessage.rw = true;
    tempMessage.isQueued = isQueued;
    tempMessage.paramsLen = sizeof(PTPCmd);
    memcpy(tempMessage.params, (uint8_t *)ptpCmd,
           tempMessage.paramsLen);
    MessageWrite(&gUART4ProtocolHandler, &tempMessage);
    (*queuedCmdIndex)++;
    return true;
}
```

结合表 2.1和程序 2.1中的common命令，需要输入的数据为Payload里面的id、rw、isQueued、params，还有length（params的字节长度）。

现有版本中已经提供了13条命令，能完成基本的运动控制，需要更多的功能请参见《Dobot Magician通信协议》。

(3) 命令发送和接收

在Protocol文件里面，程序会查询发送缓冲区是否为空。如果不为空，程序就会把串口4的发送中断使能，并在串口4的中断程序发送命令。Demo的串口4是中断接收，接收的数据

会放在接收缓冲区。通过MessageRead(ProtocolHandler *protocolHandler, Message *message)来读取缓冲区的数据，数据会放在Message结构体的变量中。

程序 2.2 Message 结构体

```
typedef struct tagPTPCmd {  
    uint8_t ptpMode;  
    float x;  
    float y;  
    float z;  
    float r;  
}PTPCmd;
```

(4) main 函数

Main函数里面主要实现了机械臂前后100mm往复运动的功能，如要修改运动的位置直接修改gPTPCmd里面的坐标参数。如要实现高级的运动需要结合Dobot Magician通信协议增加命令来实现。

程序 2.3 main 函数

```
int main(void)  
{  
    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2);  
    SysTickInit();           //初始化时钟  
    Uart1Init(115200);       //串口 1 初始化，波特率为 115200  
    Uart4Init(115200);       //串口 4 初始化，波特率为 115200  
    InitRAM();               //初始化运动参数  
    ProtocolInit();          //初始化协议  
    //配置笛卡尔坐标的运动参数  
    SetPTPCoordinateParams(&gPTPCoordinateParams, true, &gQueuedCmdIndex);  
    //配置 PTP 运动参数比值（0-100）  
    SetPTPCCommonParams(&gPTPCCommonParams, true, &gQueuedCmdIndex);  
    printf("\r\n=====Enetr demo application===== \r\n");  
    for(;;)  
    {  
        static uint32_t timer = gSysTick;  
        static uint32_t count = 0;  
        if(gSysTick - timer > 3000)           //延时 3S  
        {  
            timer = gSysTick;  
            count++;  
        }  
    }  
}
```

```
    if(count & 0x01)
    {
        //配置 PTP 运动 X 坐标+100
        gPTPCmd.x += 100;
        //运动 PTP 运动，坐标为 gPTPCmd 结构体里面的坐标
        SetPTPCmd(&gPTPCmd, true, &gQueuedCmdIndex);
    }
    else
    {
        //配置 PTP 运动 X 坐标-100
        gPTPCmd.x -= 100;
        //运动 PTP 运动，坐标为 gPTPCmd 结构体里面的坐标
        SetPTPCmd(&gPTPCmd, true, &gQueuedCmdIndex);
    }
}
ProtocolProcess();
}
```

2.3 Arduino Demo

2.3.1 硬件说明

本 Demo 是基于 ArduinoMega2560 芯片编写，因此使用本程序时需要用户自行配备一块 arduinoMega2560 开发板（理论上其他型号的 Arduino 也可运行但未经测试需要用户自行移植）。

该 Demo 中，通讯的端口使用了机械臂扩展 10PIN 接口，接口的类型为 FC-10P，接口定义如下图 2.3 所示。该 Demo 中，我们需要使用 RX、TX、GND 三个端口，机械臂与开发板连接对应关系：RX->TX1，TX->RX1，GND->GND，如图 2.4 所示。

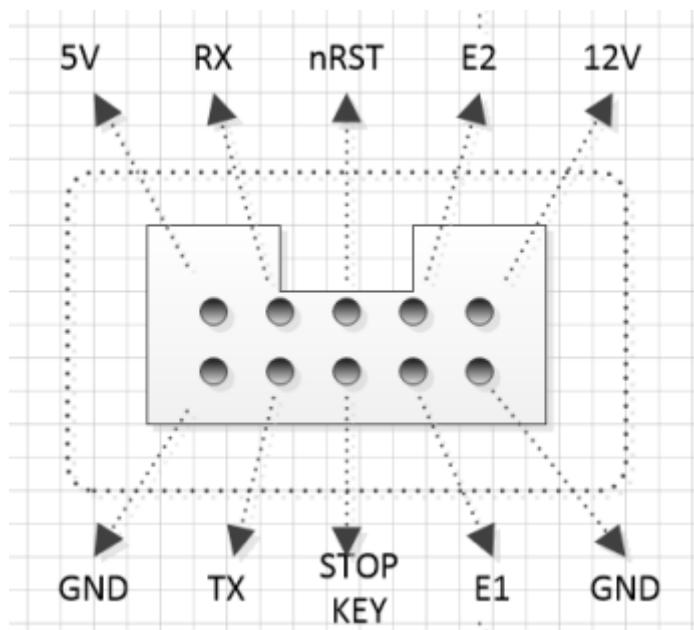


图 2.3 机械臂的扩展接口定义



图 2.4 机械臂与开发板连接

2.3.2 工程说明

本工程使用的编译器软件为Arduino 1.8.1。

(1) 通讯

Dobot协议指令由包头、负载帧长、负载帧、校验组成。

- 两个0xAA包头
- 参数长度（取值为2+N Params的长度）
- 命令索引ID
- Ctrl位（包括RW读写标记以及isQueued队列标记）
- 命令参数Params（不定长N）
- 校验和Checksum

通信协议详细内容请参见《Dobot Magician 通信协议》。

表 2.2 通讯协议格式

Header	Len	Payload				Checksum
		ID	Ctrl		Params	
			rw	isQueued		
0XAA 0XAA	2+N	XX	1/0	1/0	N（bite）	Payload Checksum

Dobot控制器支持两种类型的指令：立即指令与队列指令。

- 立即指令：Dobot控制器在收到指令后立即处理该指令，而不管当前控制器是否在还在处理其他指令。
- 队列指令：Dobot控制器在收到指令后会将该指令放入控制器内部的指令队列中，Dobot控制器将顺序执行指令。

(2) 文件结构

工程文件主要包含两部分：一是协议层处理文件包括：以 Protocol、Message、Packet 这三个开头的文件。二是应用程序实现文件包括：Common、DobotDemo 文件。此外 FlexTimer2 是 Arduino 的扩展库，用于实现定时器的功能。

2.4 iOS Demo

2.4.1 工程说明

在本Demo中，DOBOTKit.framework是静态库，把静态库加入工程即可使用，DOBOTKit是示例工程，里面展示了基于静态库的简单使用。

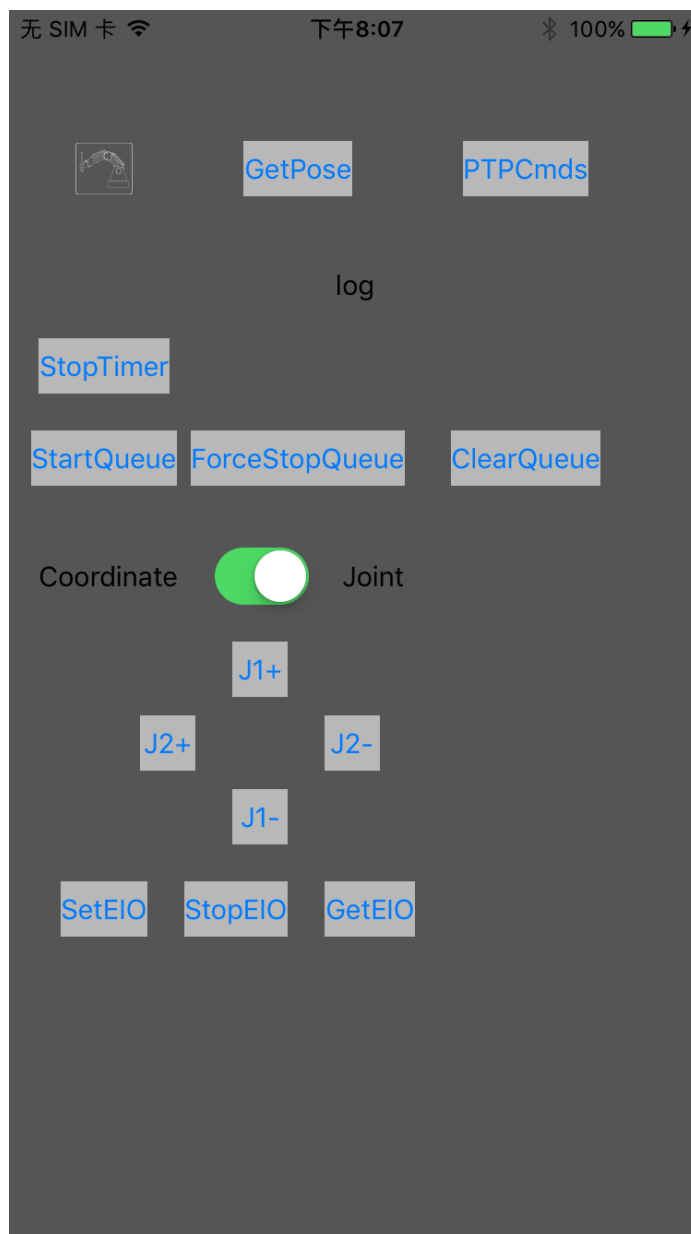


图 2.5 Demo 界面

2.4.2 代码说明

本例程实现了获取实时位姿功能，开发者可以参照本例程和通信协议文档实现其他功能，iOS静态库中已经封装好对应的API。

(1) 初始化。

使用时，首先初始化BLEMsgMgr，把当前VC作为代理和消息处理者，该单例类负责处理蓝牙连接、消息收发等。

程序 2.4 初始化 BLEMsgMgr

```
[BLEMsgMgr sharedMgr].delegate = self;  
[[BLEMsgMgr sharedMgr] addMsgHandler:self];
```

把当前ViewController加入消息处理者，这样就可以收到消息回调的通知（也可以在闭包里处理消息回调）。

(2) 蓝牙连接和断开连接。

程序 2.5 连接控制

```
if ([[BLEMsgMgr sharedMgr] isConnected]) {
    //断开连接
    [[BLEMsgMgr sharedMgr] disconnect];
}
else
{
    [[BLEMsgMgr sharedMgr]
        scanDevice:30.0f
        mode:BLESearchMode_FindAndConnectTheFirst];
}
```

连接蓝牙时，应用会连接第一个搜索到的机械臂。

(3) 消息示例，获取实时位姿。

如程序 2.6，构建一个Payload对象，设置相应的参数，调用BLEMsgMgr的sendMsg方法，就可以通过蓝牙把命令下发到机械臂。

程序 2.6 下发命令

```
Payload *payload = [[Payload alloc] init];
[payload cmdGetPose];
payload.complete = ^(MsgResult result, id msg){
    if (result == MsgResult_Ok) {
        //解析返回的位置信息
        Payload *msgPayload = ((DobotMagicianMsg *)msg).payload;
        Pose p;
        [msgPayload.params getBytes:&p length:sizeof(p)];
        NSString *text = [NSString stringWithFormat:
            @"Pose:x:%.0f,y:%.0f,z:%.0f,r:%.0f",
            p.x,p.y,p.z,p.r];
        dispatch_async(dispatch_get_main_queue(), ^{
            _lblLog.text = text;
        });
    }
};
[[BLEMsgMgr sharedMgr] sendMsg:payload];
```

如程序 2.7实现MsgHandler协议，就可以在handleMsg这个回调方法里收到机械臂返回的消息。也可以像上面的代码，实现payload.complete，在闭包里处理消息返回。

程序 2.7 接收返回数据

```

-(void)handleMsg:(DobotMagicianMsg *)msg
{
    Payload *payload = msg.payload;
    switch ((int)payload.ID) {
        case ProtocolGetPose:{
            Pose p;
            [payload.params getBytes:&p length:sizeof(p)];
            NSString *text = [NSString stringWithFormat:
                                @"Pose:x:%.0f,y:%.0f,z:%.0f,r:%.0f",
                                p.x,p.y,p.z,p.r];

            //记录当前 pose
            _lblLog.text = text;
        }
        break;
        default:
            break;
    }
}

```

2.4.3 代码说明

(1) ProtocolProcess 函数说明

该Demo的发送和接收命令均保存在Ringbuffer里面，通过ProtocolProcess函数来处理命令数据。

(2) Common 命令解析

用户需处理的两个文件是 DobotDemo.ino 文件和 common.cpp 文件。DobotDemo.ino 是主函数文件，common 是命令处理文件。以一个典型的 PTP 命令来说明，SetPTPCmd 函数传入的一个 PTPCmd 的结构体数据，队列标记，以及索引编号（暂未使用，主要用来记录当前下发的命令的序列编号）。

程序 2.8 SetPTPCmd 示例

```

int SetPTPCmd(PTPCmd *ptpCmd, bool isQueued, uint64_t *queuedCmdIndex)
{
    Message tempMessage;

    memset(&tempMessage, 0, sizeof(Message));

```



```

tempMessage.id = ProtocolPTPCmd;
tempMessage.rw = true;
tempMessage.isQueued = isQueued;
tempMessage.paramsLen = sizeof(PTPCmd);
memcpy(tempMessage.params, (uint8_t *)ptpCmd, tempMessage.paramsLen);
MessageWrite(&gUART4ProtocolHandler, &tempMessage);
(*queuedCmdIndex)++;
return true;
}

```

结合

表 2.2和程序 2.8中的common命令需要输入的数据为Payload里面的id、rw、isQueued、params，还有length（params的字节长度）。

现有版本中已经提供了 13 条命令，能完成基本的运动控制，需要更多的功能请参见《Dobot Magician 通信协议》。

（3）命令发送和接收

在Protocol文件里面，程序会查询发送缓冲区是否为空。如果不为空，程序会把从缓冲区中取出数据往串口1发送。Demo的串口1是中断接收，接收的数据会放在接收缓冲区。通过MessageRead(ProtocolHandler *protocolHandler, Message *message)来读取缓冲区的数据，数据会放在Message结构体的变量中。

程序 2.9 Message 结构体

```

typedef struct tagPTPCmd {
uint8_t ptpMode;
    float x;
    float y;
    float z;
    float r;
}PTPCmd;

```

（4）配置函数说明

1. 硬件初始化。

程序 2.10 setup 函数

```

void setup() {
    Serial.begin(115200);           //启动串口 0，波特率为 115200
    Serial1.begin(115200);         //启动串口 1，波特率为 115200
    printf_begin();                 //配置 Printf，定向输出到串口 0
}

```

```
//Set Timer Interrupt
FlexiTimer2::set(100,Serialread); //配置定时中断，每 100ms 执行 Serialread 函数
FlexiTimer2::start();              //启动定时器
}
```

2. Serialread函数读取串口1的数据并放到接收缓冲区中。

程序 2.11 Serialread()函数

```
void Serialread()
{
    while(Serial1.available()) {          //判断串口 1 是否有数据
        uint8_t data = Serial1.read();    //读取 8bit 数据存放到 data
        if (RingBufferIsFull(
            &gSerialProtocolHandler.rxRawByteQueue)
            == false) {
            //RingBuffer 有空余空间则会保存数据
            RingBufferEnqueue(
                &gSerialProtocolHandler.rxRawByteQueue,
                &data);
        }
    }
}
```

3. Serial_putc(char c, struct __file *)和printf_begin(void)主要实现Printf功能。
4. InitRAM(void)主要实现配置默认的运动参数。

(5) 主循环函数

主循环函数里面主要实现了机械臂前后100mm往复运动的功能,如要修改运动的位置直接修改gPTPCmd里面的坐标参数。如要实现高级的运动需要结合通信协议增加命令来实现。

程序 2.12 main 函数

```
int main(void)
{
    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2);
    SysTickInit();          //初始化时钟
    Uart1Init(115200);       //串口 1 初始化，波特率为 115200
    Uart4Init(115200);       //串口 4 初始化，波特率为 115200
    InitRAM();               //初始化运动参数
    ProtocolInit();          //初始化协议
}
```

```
//配置笛卡尔坐标的运动参数
SetPTPCoordinateParams(&gPTPCoordinateParams,
                        true,
                        &gQueuedCmdIndex);

//配置 PTP 运动参数比值（0-100）
SetPTPCoordinateParams(&gPTPCoordinateParams, true, &gQueuedCmdIndex);

printf("\r\n=====Enetr demo application===== \r\n");
for(;;)
{
    static uint32_t timer = gSystick;
    static uint32_t count = 0;
    if(gSystick - timer > 3000)        //延时 3S
    {
        timer = gSystick;
        count++;
        if(count & 0x01)
        {
            gPTPCmd.x += 100;           //配置 PTP 运动 X 坐标+100

            //运动 PTP 运动，坐标为 gPTPCmd 结构体里面的坐标
            SetPTPCmd(&gPTPCmd,
                      true,
                      &gQueuedCmdIndex);
        }
        else
        {
            //配置 PTP 运动 X 坐标-100
            gPTPCmd.x -= 100;

            //运动 PTP 运动，坐标为 gPTPCmd 结构体里面的坐标
            SetPTPCmd(&gPTPCmd, true, &gQueuedCmdIndex);
        }
    }

    ProtocolProcess();//Protocol 进程
}
```

2.5 Android Demo

2.5.1 工程说明

Dobot.jar 是DobotMagician的封装库，里面封装了Android4.3+平台的BLE常用操作及DobotMagician通信协议的部分封装。使用前只需导入Dobot.jar到Android Studio(或Eclipse)项目中的libs目录便可在工程中引用封装好的API对Dobot Magician进行操作。DobotDemo是示例工程，简单的演示了如何调用Dobot.jar中API。

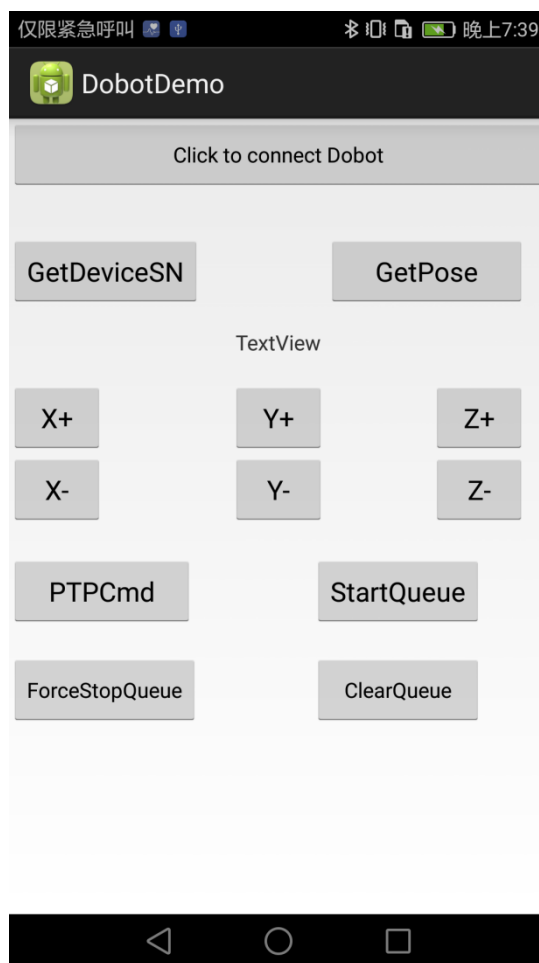


图 2.6 Andriod Demo 界面

2.5.2 代码说明

本例程实现了获取实时位姿功能，开发者可以参照本例程和通信协议文档实现其他功能，Dobot.jar库中已经封装好部分对应的API。

- (1) 在 Android Project 的 AndroidManifest.xml 文件中添加蓝牙权限。

程序 2.13 添加蓝牙权限

```
<uses-permission
    android:name="android.permission.BLUETOOTH"/>
</uses-permission>
```

```

        android:name="android.permission.BLUETOOTH_ADMIN"/>
<uses-feature
    android:name="android.hardware.bluetooth_le"
    android:required="true" />

```

- (2) 创建 Dobot 对象。

程序 2.14 创建 Dobot 对象

```

//创建时传入 Context 参数，实现 DobotCallbacks()接口
Dobot myDobot = new Dobot(this,new DobotCallbacks() {
    @Override
    public void DobotDisconnected(BluetoothGatt arg0, BluetoothDevice arg1) {
        // TODO Auto-generated method stub
        Log.d("dobot","dobot Disconnected");
    }

    @Override
    public void DobotConnected(BluetoothGatt arg0, BluetoothDevice arg1) {
        // TODO Auto-generated method stub
        Log.d("dobot","dobot connected");
    }

    @Override
    public void DobotConnectTimeOut() {
        // TODO Auto-generated method stub
        Log.d("dobot","dobot connect timeout");
    }
});

```

- (3) 初始化 Dobot 对象。

程序 2.15 初始化 Dobot 对象

```

myDobot.initialize();

```

- (4) 连接手机至 Dobot Magician。

程序 2.16 连接

```

myDobot.Connect(); //需要断开连接时调用 myDobot.close();

```

- (5) 调用 API 获取实时位姿。

程序 2.17 获取位姿

```
myDobot.GetPose(new DataReceiveListener() {  
    @Override  
    public void OnReceive() {  
        // TODO Auto-generated method stub  
        TagPose pose = myDobot.ReadPose();  
        float x= pose.getX();  
        float y= pose.getY();  
        float z= pose.getZ();  
        float r= pose.getR();  
        Log.d("dobot", "X :"+x+"---Y :"+y+"---Z :"+z+"---R :"+r);  
    }  
});
```